

[← Go Back](#)

Hardware

GIGA Display Shield



Tutorials

[Getting Started with the GIGA Display Shield](#)[GIGA Display Shield GFX Guide](#)[GIGA Display Shield LVGL Guide](#)[GIGA Display Shield Draw Images Guide](#)[GIGA Display Shield Touch Interface](#)[GIGA Display Shield Automatic Screen Orientation with IMU](#)[GIGA Display Shield Microphone Guide](#)[GIGA Display Shield Camera Connector Guide](#)[GIGA Display Shield 3D Engine Tutorial](#)[SquareLine Studio With the GIGA Display Shield](#)[Display Camera Feed on GIGA Display Shield with OpenMV](#)[GIGA Display Shield emWin Guide](#)[GIGA Display Shield AppWizard Guide](#)[GIGA Display Shield Display Image From USB](#)[Home](#) / [Hardware](#) / [GIGA Display Shield](#) / [GIGA Display Shield LVGL Guide](#)

GIGA Display Shield LVGL Guide

Learn how to use LVGL with the GIGA Display Shield.

Author · Benjamin Dannegård · Last revision · 12/05/2024

Introduction

LVGL is a graphical framework for building powerful UIs, and is fully compatible with the GIGA Display Shield. It allows you to build UIs, using pre-made widgets like buttons, images, loading bars, sliders, checkboxes, etc. It also allows you to fully customize the screenspace on the display. In this guide, we will go through some of the different components, so you can learn how to best implement it in your projects.



LVGL is a large framework, and the aim of this guide is to get you familiar with it. For much more information, visit the [LVGL documentation](#).

Hardware & Software Needed

[Arduino GIGA R1 WiFi](#)[Arduino GIGA Display Shield](#)[Arduino IDE](#)

ON THIS PAGE

Introduction

[Hardware & Software Needed](#)[Downloading the Library and Board Package](#)[General Set Up](#) +[Visual Elements](#) +[Functional Elements](#) +[Conclusion](#)[Next Step](#)[Help](#)

Downloading the Library and Board Package

The GIGA R1 Board Package includes the [Arduino_H7_Video](#) library that handles the display.

In this guide, we will be using three different libraries:

[Arduino_H7_Video](#), this one is bundled with the Board Package, so make sure you have the latest version of the [GIGA Board Package](#) installed.

[Arduino_GigaDisplayTouch](#)

[lvgl](#)

To install them, open the library manager and install the latest version by searching for **"Arduino_GigaDisplayTouch"** and **"lvgl"**.

In the sketch include the libraries like this:

```
1 #include "Arduino_H7_Vide
2 #include "lvgl.h"
3 #include "Arduino_GigaDis
```

General Set Up

In this section, we will go through the fundamental elements of an LVGL sketch:

How to define & configure the display,

how to create a grid layout

how to update the display.



At the end of this section, you will find a complete example which implements a grid layout. You can [skip directly to the example](#), but if you have no prior experience with LVGL, we recommend you follow the information below.

Display Shield Configuration

The first thing we need to do is to define the display we are using, by creating a object using the `Arduino_H7_Video` class, specifying the height, width and type of display.

```
1 Arduino_H7_Video
```

And if you want to use touch with your application, we need to create an object using the `Arduino_GigaDisplayTouch` class.

```
1 Arduino_GigaDisplayTouch
```

In the `setup()`, we begin by initializing the display and the touch detector.

```
1 void setup(){  
2   delay(3000);  
3   Display.begin();  
4   TouchDetector.begin();
```

With these things set up, we can go on to configure the LVGL elements.

LVGL Screen Configuration

When creating elements, information about the screen and placement needs to be provided. Let's create a pointer variable that can be used whenever the screenspace needs to be used. The pointer variable will be named `screen` and to use the current screen for the pointer use `lv_scr_act()`. This code should also be added inside `setup()`.

```
1 lv_obj_t * screen = lv_ob
```

The size of the screen space needs to be set for the pointer that is declared. The size can be set to anything within the display's size parameters. To make it easy we can use the entire size:

```
1 lv_obj_set_size(screen, D
```

Creating a Grid Layout

Creating a grid that you can then fill with elements will consist of a defined column and row. This

```
col_dsc[] = {370, 370,  
LV_GRID_TEMPLATE_LAST};
```

will create two columns with 370 px width. To add more columns simply

```
col_dsc[] = {100, 100, 100,
100, LV_GRID_TEMPLATE_LAST};
```

, this will create four columns with 100 px width. The same logic is applied to the row definition.

```
1 static lv_coord_t col_dsc
2 static lv_coord_t row_d
```

Then like before a pointer for the screenspace needs to be created. Here it will be called `grid`.

```
1 lv_obj_t * grid = lv_obj_
```

To set the grid description that we defined before use:

```
1 lv_obj_set_grid_dsc_array
```

Now that the columns and rows have been defined the overall screen needs to be taken into account. This is achieved by using

```
lv_obj_set_size(grid,
Display.width(),
Display.height())
```

, to make it easy we will allow the `grid` to use the entire screen.

```
1 lv_obj_set_size(grid, Dis
```

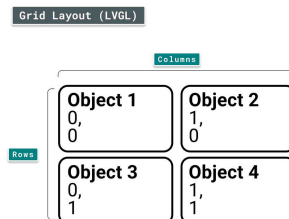
Then, just to test it out, let's add an object to the grid:

first declare the `obj` pointer
add it to the `grid`,
then we set the grid cell via
`lv_obj_set_grid_cell()`
method.

```
1 lv_obj_t* obj;  
2 obj = lv_obj_create(grid)  
3 lv_obj_set_grid_cell(obj,  
4
```

The `lv_obj_set_grid_cell()` have a set of parameters related to how the object is positioned and aligned. In this case, we have created a 2x2 grid, where the `0` (column) and `0` (row) indicates where on the grid it should go.

Take a look at this graphic to understand it better:



How grids work in LVGL.

Update Loop

Include this in the loop of your sketch to make sure the LVGL engine is running and updating the screen. The `lv_timer_handler()` function should be called at max every 5 milliseconds. If it would be called more frequently than that please use a non-blocking method to delay it.

```
1 void loop() {  
2   lv_timer_handler();  
3 }
```

Minimal Example

The summary of all code snippets in this section can be found in the example below. It implements a 2x2 grid, as well as adds an object in each slot.

```
1 #include "Arduino_H7_Vi  
2 #include "lvgl.h"  
3 #include "Arduino_GigaL  
4  
5 Arduino_H7_Video Displa  
6 Arduino_GigaDisplayTouc  
7  
8 void setup() {  
9   delay(3000);  
10  Display.begin();  
11  TouchDetector.begin()  
12  
13  //Display & Grid Setu  
14  lv_obj_t* screen = lv  
15  lv_obj_set_size(scree  
16  
17  static lv_coord_t col  
18  static lv_coord_t row  
19  
20  lv_obj_t* grid = lv_o  
21  lv_obj_set_grid_dsc_a  
22  lv_obj_set_size(grid,  
23  
24  //top left  
25  lv_obj_t* obj;  
26  obj = lv_obj_create(g  
27  lv_obj_set_grid_cell(  
28  
29
```

Visual Elements

To display an image on the screen we first need to define what that image that should be. Take the desired image, **convert it into the correct format** and place the image in the same folder as the sketch.

Now use

`LV_IMG_DECLARE(filename);`. For example the image we use will be named `img_arduinologo`.

```
1 LV_IMG_DECLARE(img_arduin
```

`obj` will be a pointer that will be used to hold the information about the screenspace information for the image. The `img1` pointer will be used for the elements of the image itself.

```
1 lv_obj_t * obj;  
2 lv_obj_t * img1;
```

Then create the image object with `obj` as a parent. Then we can link the image and image pointer together.

```
1 img1 = lv_img_create(obj)  
2 lv_img_set_src(img1, &i
```

To make sure we see the image use the align function to make it centered. Then at last set the size of image with

```
lv_obj_set_size(img1, WIDTH,  
HEIGHT)
```



```
1 lv_obj_align(img1, LV_ALI
2 lv_obj_set_size(img1, 2
```



An image rendered on the
GIGA Display Shield with
LVGL

Full Example:

```
1 #include "Arduino_H7_Vi
2 #include "lvgl.h"
3
4 Arduino_H7_Video
5
6 void setup() {
7     delay(3000);
8     Display.begin();
9
10    lv_obj_t * screen = l
11    lv_obj_set_size(scree
12
13    static lv_coord_t col
14    static lv_coord_t row
15
16    lv_obj_t * grid = lv_
17
18    lv_obj_set_grid_dsc_a
19
20    lv_obj_set_size(grid,
21
22    lv_obj_center(grid);
23
24    lv_obj_t * obj;
25    lv_obj_t * img1;
26
27    obj = lv_obj_create(g
28    lv_obj_set_grid_cell(
```

Displaying text is usually done with labels. These are used in most of the other examples to put text on buttons and switches. Like any other element you first have to create the object with:

```
1 lv_obj_t * label;
```

Then the labels text can be set with:

```
1 lv_label_set_text(label,
```

If you want to insert a variable that is not a string, use:

```
1 //This will make the label  
2 lv_label_set_text_fmt(label,
```

To use bigger font sizes it has to be enabled in the `lv_conf.h` file. This file can be found in the **mbd_giga/libraries/Arduino_H7_Video/src** folder. Find the **FONT USAGE** section, here you can see all the font sizes. If you want to enable any size simply change the `0` next to any of the font sizes into a `1`. Like this:



Functional Elements

Checkbox

`obj` will be a pointer that will be used to hold the information about the screenspace information for the checkbox. The `checkbox` pointer will be used for the elements in the checkbox itself.

```
1 lv_obj_t * obj;  
2   lv_obj_t * checkbox;
```

Assign the screenspace info to `obj`, that was detailed in the [Screen Configuration](#) section. To create the checkbox object use

`lv_checkbox_create(obj)` and

assign it to a suitable variable, here we use the `checkbox` pointer. Next set the text that will appear next to the checkbox by using

```
lv_checkbox_set_text(checkbox,  
"Example");
```

, here `Example` will be printed next to the checkbox.

```
1 obj = lv_obj_create(scre  
2   checkbox = lv_checkbox_  
3   lv_checkbox_set_text(ch
```

The startup state of the checkbox can be set with

`lv_obj_add_state()`. Where the object and state has to be specified:



Checkboxes rendered on
the GIGA Display Shield
with LVGL

Full Example:

```
1 #include "Arduino_H7_Vi
2 #include "lvgl.h"
3 #include "Arduino_GigaL
4
5 Arduino_H7_Video
6 Arduino_GigaDisplayTouc
7
8 void setup() {
9     delay(3000);
10    Display.begin();
11    TouchDetector.begin()
12
13    lv_obj_t * screen = l
14    lv_obj_set_size(scree
15
16    static lv_coord_t col
17    static lv_coord_t row
18
19    lv_obj_t * grid = lv_
20
21    lv_obj_set_grid_dsc_a
22
23    lv_obj_set_size(grid,
24
25    lv_obj_center(grid);
26
27    lv_obj_t * obj;
28
```

Radio Button

A radio button is created in the same way as a checkbox, but with some

elements will allow for them to be added to the checkbox options.

```
1 static lv_style_t style_r  
2 static lv_style_t style
```

Now initialize the style variable that was set in the previous step:

```
1 lv_style_init(&style_radi
```

The size of the radio button is set with `lv_style_set_radius`. To make the radio button checkable use `lv_style_init(&style_radio_chk)`. And the color or background of the filled radio check can be set with `lv_style_set_bg_img_src`.

```
1 lv_style_set_radius(&styl  
2 lv_style_init(&style_ra  
3 lv_style_set_bg_img_src
```



Radio buttons rendered
on the GIGA Display
Shield with LVGL

Full Example:

```
1 #include "Arduino_H7_Vi
2 #include "lvgl.h"
3 #include "Arduino_GigaL
4
5 Arduino_H7_Video
6 Arduino_GigaDisplayTouc
7
8 void setup() {
9     delay(3000);
10    Display.begin();
11    TouchDetector.begin()
12
13    lv_obj_t * screen = l
14    lv_obj_set_size(scree
15
16    static lv_coord_t col
17    static lv_coord_t row
18
19    lv_obj_t * grid = lv_
20
21    lv_obj_set_grid_dsc_a
22
23    lv_obj_set_size(grid,
24
25    lv_obj_center(grid);
26
27    lv_obj_t * obj;
28
```

Slider

`obj` will be a pointer that will be used to hold the information about the screenspace information for the slider. The `slider` pointer will be used for the elements of the slider itself. The `label` pointer will be used for the text that will be attached to the slider.

```
1 lv_obj_t * obj;
2 lv_obj_t * slider;
3 lv_obj_t * label;
```

Now the slider can be created with:

```
1 slider = lv_slider_create
```

Now the value of the slider needs to be defined, here the max value of the slider will be `75` and the animation will be default set as off as it is only needed when it is interacted with.

```
1 lv_slider_set_value(slide
```

If you want a label by your slider it can be created like you would create any other label. Using `lv_obj_align_to` allows for the label to be attached to the slider element. Changing the `LV_ALIGN_OUT_BOTTOM_MID` to determine where the text will be relative to the slider. You can find all the different options for alignment [here](#).

```
1 label = lv_label_create(o
2   lv_label_set_text(label
3   lv_obj_align_to(label,
```



Slider rendered on the
GIGA Display Shield with
LVGL

```
1 #include "Arduino_H7_Vi
2 #include "Arduino_GigaL
3
4 #include "lvgl.h"
5
6 Arduino_H7_Video
7 Arduino_GigaDisplayTouc
8
9 void setup() {
10     delay(3000);
11     Display.begin();
12     TouchDetector.begin()
13
14     lv_obj_t * screen = l
15     lv_obj_set_size(scree
16
17     static lv_coord_t col
18     static lv_coord_t row
19
20     lv_obj_t * grid = lv_
21
22     lv_obj_set_grid_dsc_a
23
24     lv_obj_set_size(grid,
25
26     lv_obj_center(grid);
27
28     lv_obj_t * label;
```

Bar

To make a bar, like a loading bar, we need to include some animation. Let's first set up the slider itself and then move on to the animation.

`obj` will be a pointer that will be used to hold the information about the screenspace information for the bar. The `bar` pointer will be used for the elements of the bar itself.

```
1 lv_obj_t * obj;
2 lv_obj_t * bar;
```



```
1 bar = lv_bar_create(obj);
```

Set the desired size of the bar with `lv_obj_set_size`. The value of the bar needs to be defined, here the max value of the bar will be `70` and the animation will be default set as off.

```
1 lv_obj_set_size(bar, 200,  
2   lv_bar_set_value(bar, 7
```

Now for the animation. First create the slider variable and initialize it:

```
1 lv_anim_t animation;  
2   lv_anim_init(&animation
```

The animation time needs to be defined. It can be set with `lv_anim_set_time` which sets the duration of the animation and `lv_anim_set_playback_time` which makes the animation play back to when the forward direction is ready. The animation variable and the time in milliseconds has to be defined.

```
1 lv_anim_set_time(&animati  
2   lv_anim_set_playback_ti
```

To connect the animation to the bar use:

```
1 lv_anim_set_var(&animatio
```

The start and end values of the animation has to be set, here they are `0` and `100` respectively.

```
1 lv_anim_set_values(&anima
```

How many times the animation will repeat can also be set, with this code the animation will repeat forever. And then at last we can create the animation with `lv_anim_start`.

```
1 lv_anim_set_repeat_count(  
2   lv_anim_start(&animatio
```

When the bar animates we can set it so that a separate callback function will be called. Here that function will be named `set_bar_val`.

```
1 lv_anim_set_exec_cb(&anim
```

In this separate callback function the bar value will be reset and the animation will be turned on again.

```
1 static void set_bar_val(v
2   lv_bar_set_value((lv_ob
3 }
```



Full Example:

```
1 #include "Arduino_H7_Vi
2 #include "Arduino_Giga
3
4 #include "lvgl.h"
5
6 Arduino_H7_Video
7 Arduino_GigaDisplayTouc
8
9 static void set_slider_
10   lv_bar_set_value((lv_
11 }
12
13 void setup() {
14   delay(3000);
15   Display.begin();
16   TouchDetector.begin()
17
18   lv_obj_t * screen = l
19   lv_obj_set_size(scree
20
21   static lv_coord_t col
22   static lv_coord_t row
23
24   lv_obj_t * grid = lv_
25
26   lv_obj_set_grid_dsc_a
27
28   lv_obj_set_size(grid,
```

A button will need two parts, the design of the button itself and the callback event function which determines what happens when the button is pressed. Let's start with designing the button.

`obj` will be a pointer that will be used to hold the information about the screenspace information for the button. The `button` pointer will be used for the elements in the button itself. The `label` pointer will be used for the text that will be put on the button.

```
1 lv_obj_t * obj;  
2   lv_obj_t * button;  
3   lv_obj_t * label;
```

Now the button can be created with:

```
1 button = lv_btn_create(obj
```

Set the size of the button with

```
lv_obj_set_size(btn, WIDTH,  
HEIGHT)
```

. For example:

```
1 lv_obj_set_size(btn, 100,
```

Let's make the label on the button a child of the button by using

```
label = lv_label_create(button)
```

. Then the label can be set to whatever text it needs to be and center the text on top of the button

will now say `Click me!` at the center of it.

```
1 label = lv_label_create(b
2   lv_label_set_text(label
3   lv_obj_center(label);
```

When the button is clicked we need to assign it a function to execute, let's call this function `button_event_callback`. Assign it to the correct button and set it to be executed when the button is clicked with `LV_EVENT_CLICKED`.

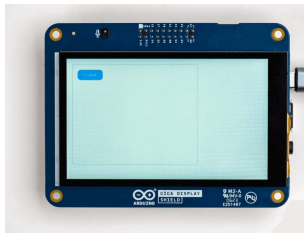
```
1 lv_obj_add_event_cb(butto
```

Now let's create the callback function that will be called when the button is clicked. By creating pointers that will point to the original elements we can change them easily in our function. This function will make it so that when the button is clicked the label text on the button will be changed to `Clicked!`.

```
1 static void button_event_
2   lv_obj_t * button = lv_
3   lv_obj_t * label = lv_o
4   lv_label_set_text_fmt(1
5 }
```



A button rendered on the
GIGA Display Shield with
LVGL



Button when it has been
pressed

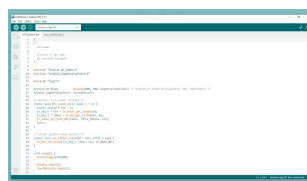
Full Example:



```
1 #include "Arduino_H7_Vi
2 #include "lvgl.h"
3 #include "Arduino_GigaL
4
5 Arduino_H7_Video
6 Arduino_GigaDisplayTouc
7
8 static void btn_event_c
9     lv_obj_t * btn = lv_e
10     lv_obj_t * label = lv
11     lv_label_set_text_fmt
12 }
13
14 void setup() {
15     delay(3000);
16     Display.begin();
17     TouchDetector.begin()
18
19     lv_obj_t * screen = l
20     lv_obj_set_size(scree
21
22     static lv_coord_t col
23     static lv_coord_t row
24
25     lv_obj_t * grid = lv_
26
27     lv_obj_set_grid_dsc_a
28
```

Conclusion

This guide went through the building blocks of the different components that can be implemented with LVGL. To see these examples in a full running example sketch go to **File > Examples > Arduino_H7_Video > LVGLDemo**.



Example in the IDE

This example sketch will show the

Next Step

LVGL has a lot of customizability, if you are interested in playing around more with this, you can find many different examples on the official website for [LVGL](#). These can easily be put in a sketch for the GIGA Display Shield, just remember to use the display-specific configuration that was shown at the [start of this tutorial](#).

The GIGA Display Shield features some on-board sensors, like an IMU and a microphone. To see how these can be used with LVGL check out our [Orientation tutorial](#), which will show you how to use the on-board IMU to rotate an image. Or have a look at our [Microphone tutorial](#), which will show you how to merge microphone readings with LVGL components.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository. If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

